

Dependability analysis of a Java open-source project

Matthias Gerhard

m.gerhard@studenti.unisa.it
Università degli Studi di Salerno
Salerno, Campania, Italy

Max Timur Sönmez

m.soenmez@studenti.unisa.it
Università degli Studi di Salerno
Salerno, Campania, Italy

Abstract

This paper documents the systematic hardening of an existing Spring Boot microservice application using static analysis, mutation testing, and DevSecOps integration. The intervention successfully achieved 100% code coverage and a 73% performance gain by optimizing a critical algorithmic bottleneck via microbenchmarks. However, the study confirmed that rigorous formal specification (JML) is incompatible with modern, Lombok-heavy architectures due to bytecode generation conflicts. We conclude by defining a pragmatic, hybrid strategy for software dependability centered on a rigorous, security-aware CI/CD pipeline.

Keywords

Software Dependability, Mutation Testing, Static Analysis, PiTest, JaCoCo, Spring Boot, JML

1 Introduction

Modern software development increasingly relies on complex microservice architectures, often utilizing robust frameworks such as Spring Boot and event-streaming platforms like Apache Kafka. While these technologies accelerate functional development, they often introduce complexity that makes rigorous quality assurance and formal verification difficult. Open-source repositories frequently demonstrate functional correctness but lack the software dependability layers—such as mutation testing, strict static analysis, and supply chain security—required for production-grade systems.

This paper presents a case study on retrofitting dependability measures onto an existing open-source artifact. We selected and forked the `ecommerce-api-spring-kafka` repository [1], a representative microservices application implementing an e-commerce backend. While the original repository provided a functional implementation of shopping cart and order processing logic, it lacked a comprehensive automated testing strategy, formal behavioral specifications, or security auditing.

Our work follows a systematic approach to software hardening, addressing the following key areas:

- (1) **Environmental Stabilization:** We addressed immediate build reproducibility issues by upgrading the Docker base image to `eclipse-temurin:17-jdk` and pinning Kafka/Zookeeper versions to resolve startup failures in the containerized environment.
- (2) **Automated Quality Gates:** We established a continuous integration (CI) pipeline using GitHub Actions to enforce build verification, unit testing, and artifact delivery (Docker Hub) on every push.

- (3) **Static Analysis & Mutation Testing:** We integrated a suite of static analysis tools (SpotBugs, PMD, Checkstyle) to establish a quality baseline and applied Mutation Testing (PiTest) to identify and repair gaps in the test suite’s semantic coverage.
- (4) **Formal Specification:** We applied Java Modeling Language (JML) annotations to core services to formalize system behavior, utilizing them as strict documentation to identify architectural constraints imposed by framework-heavy designs (e.g., Lombok).
- (5) **Performance & Security Hardening:** We extended the dependability scope beyond functional logic by establishing performance baselines via JMH microbenchmarks and integrating a DevSecOps layer (Snyk, GitGuardian, SonarCloud) to audit dependencies and harden the software supply chain.

The remainder of this paper details the methodology used to integrate these tools, presents the empirical metrics gathered (coverage improvements, mutation scores, vulnerability reports), and discusses the trade-offs between strict formal verification and pragmatic automation in modern Java ecosystems.

2 Dependability Goals

The primary objective of this study was to transform a functional prototype into a robust, maintainable, and verifiable software artifact. We defined six specific dependability goals to guide our interventions:

- **Reproducibility & Stability:** The build environment must be deterministic. The original repository suffered from runtime failures due to unpinned dependencies and incompatible base images. Our goal was to ensure the application builds and launches consistently across different environments (Phase 1).
- **Code Quality Baseline:** We aimed to establish a quantitative baseline for code quality using industry-standard static analysis tools (SpotBugs, PMD, Checkstyle) to identify potential bugs, bad practices, and style violations before applying fixes (Phase 2).
- **Test Completeness & Robustness:** Merely having tests is insufficient. We aimed to increase meaningful code coverage (excluding generated code) and verify the quality of the test suite itself using Mutation Testing. The goal was to ensure that tests are sensitive enough to detect real regressions (Phase 3).
- **Behavioral Specification:** We sought to formalize the business logic of core services using the Java Modeling Language (JML). The goal was to document strict preconditions and postconditions for critical methods (e.g., `ShoppingCartService`), creating a clear contract for future development (Phase 4).

- **Performance Profiling:** Dependability includes efficiency. We aimed to establish a performance baseline for the core domain logic using microbenchmarks (JMH). This ensures that future refactoring or dependability interventions do not degrade the system's throughput or latency (Phase 5).
- **Security & Supply Chain Integrity:** A dependable system must be secure. Our goal was to integrate automated security auditing (DevSecOps) into the pipeline to detect hardcoded secrets, analyze dependency vulnerabilities, and harden the CI supply chain against external threats (Phase 6).

3 Methodology

Our methodology followed a phased approach, starting with environmental stabilization and progressing toward advanced verification techniques.

3.1 Phase 1: Environment Stabilization & CI

Before addressing the code, we stabilized the execution environment.

- **Containerization:** We updated the Docker base image from `openjdk:17-jdk-slim` to `eclipse-temurin:17-jdk` to resolve compatibility issues with the project's build environment.
- **Dependency Pinning:** The original `docker-compose.yml` used latest tags for Kafka and Zookeeper, causing startup failures. We pinned these services to version 3.3.1 to ensure stability.
- **Continuous Integration:** We implemented a GitHub Actions workflow that triggers on every push to the `develop` branch. This pipeline executes `mvn clean verify` to enforce that the application compiles and passes all tests automatically.

3.2 Phase 2: Static Analysis Integration

We integrated three distinct static analysis tools into the Maven build lifecycle to detect defects early.

- **Tool Selection:** We employed **SpotBugs** for bug patterns, **PMD** for programming flaws, and **Checkstyle** for adherence to coding standards.
- **Execution:** These tools were configured to run via `mvn verify`, generating XML reports (`checkstyle-result.xml`, `pmd.xml`, etc.).
- **Strategy:** We categorized findings by severity (Critical, Medium, Low) but intentionally applied no fixes at this stage. This allowed us to preserve the original state as a baseline for measuring the impact of later testing and mutation efforts.

3.3 Phase 3: Coverage & Mutation Testing

To improve the reliability of the test suite, we used a data-driven approach.

- **Coverage Analysis (JaCoCo):** We integrated JaCoCo to measure instruction and branch coverage. Crucially, we configured exclusions for generated code (e.g., `*MapperImpl.class`) to prevent false negatives in our metrics. The initial analysis

revealed a test coverage of 79.3% with 62.5% of branches tested.

- **Test Expansion:** Based on the JaCoCo reports, we extended the unit test suite, specifically targeting uncovered branches in the domain logic.
- **Mutation Testing (PiTest):** We implemented PiTest to evaluate the quality of our tests. We encountered "unkillable mutants" in Lombok-generated code (e.g., Builders, Getters/Setters) and AOP aspects. Consequently, we refined our configuration to exclude boilerplate code, focusing mutation analysis on the core domain logic where semantic meaning exists.

3.4 Phase 4: Formal Specification (JML)

In the final phase, we applied JML annotations to document the behavior of critical services.

- **Scope:** We targeted the `ShoppingCartService` and `OrderService`, defining invariants, preconditions (e.g., non-null arguments), and postconditions (e.g., total price calculations).
- **Constraints:** We found that the project's heavy reliance on Spring Boot and Lombok prevented the use of OpenJML's static verification (`esc`) and runtime checking. The "magic" bytecode generation of these frameworks obscures the plain Java logic required by JML tools. Therefore, JML served primarily as a formal documentation layer rather than an automated verification gate.

3.5 Phase 5: Performance Analysis (JMH)

To assess the computational efficiency of the domain logic, we integrated the Java Microbenchmark Harness (JMH).

- **Objective:** The benchmarking suite was designed to isolate core service methods from infrastructure latency (database/network). This process served a dual purpose: providing a performance baseline for throughput and execution time, and analytically validating the efficiency of critical framework components, such as the code generation of `MapStruct`.
- **Scope:** We established two primary benchmarks: the `ProductMapperBenchmark` to compare framework efficiency against manual mapping, and the `AddressSearchBenchmark` to identify potential algorithmic bottlenecks in the order processing logic.

3.6 Phase 6: Security Analysis & DevSecOps

To enforce reliability beyond functional correctness, we integrated automated security auditing into the pipeline.

- **Secret Detection (GitGuardian):** We integrated GitGuardian to scan the commit history for hardcoded secrets and API keys, ensuring no sensitive data was leaked into version control.
- **Vulnerability Scanning (Snyk):** We deployed Snyk to analyze project dependencies. A key methodological challenge here was resolving environment isolation; we configured the scanner to run directly on the host runner rather than inside a container, ensuring it could access the correct JDK 17 context.

- **Static Security (SonarCloud):** We added SonarCloud to detect security hotspots (e.g., CORS configurations). Additionally, we hardened the supply chain by transitioning GitHub Action dependencies from mutable version tags to immutable commit SHAs.

4 Empirical Results

This section presents the quantitative data gathered from our dependability interventions, focusing on code coverage, mutation scores, and the feasibility of formal specifications in the target environment.

4.1 Static Analysis Baseline

The integration of SpotBugs, PMD, and Checkstyle provided a quality baseline. The analysis was executed via the Maven build cycle, generating XML reports for all modules (stored in the repository in "docs/static-code-analysis-reports" folder).

The most critical findings, identified by SpotBugs, concerned potential runtime dependability issues, specifically the exposure of internal mutable object representations in multiple DTOs and in the `PaymentService`. Medium-severity findings primarily related to code style and maintainability concerns, such as the use of the `DesignForExtensionCheck` by Checkstyle and excessive static imports flagged by PMD.

These reports served as a static baseline. No automated fixes were applied at this stage to ensure that subsequent dynamic testing metrics reflected the state of the original logic rather than auto-corrected code.

4.2 Test Coverage (JaCoCo)

We utilized JaCoCo to measure the effectiveness of the unit test suite. To ensure the metrics reflected relevant business logic, we configured the tool to exclude generated artifacts, specifically `**MapperImpl.class` (MapStruct implementations) and `**/generated/**` directories.

The process was iterative: the initial JaCoCo report served as a blueprint to systematically identify and close gaps in the testing structure.

Metric	Initial Baseline	Final Coverage
Line Coverage	79.3%	100%
Branch Coverage	62.5%	100%

Table 1: Improvement in Code Coverage Metrics

The baseline revealed a significant gap in the testing of conditional logic (Branch Coverage: 62.5%). The analysis showed that while the "happy path" was often tested, error handling and edge cases in the service layer remained largely unverified.

By utilizing the initial JaCoCo output to guide the expansion of the unit test suite, we successfully addressed the missing test cases, resulting in full **100% Line Coverage** and **100% Branch Coverage** for the core domain packages.

4.3 Mutation Testing (PiTest)

To assess the quality of the test suite beyond simple line execution, we applied PiTest. The initial runs identified numerous "unkillable" mutants arising from boilerplate code (e.g., Lombok builders, getters/setters). After filtering these non-semantic mutations, we focused on the domain packages.

Table 2 summarizes the mutation coverage improvements achieved after extending the test suite based on the initial reports.

Package	Mut. Score
domain.address	25% → 100%
domain.shoppingcart	60% → 77%
infrastructure.aspect	50%
infrastructure.exceptions	60%

Table 2: Mutation Testing Results by Package

The improvement in `'domain.address'` (reaching 100%) validates that value objects, often overlooked, are easy to perfect once identified. However, the `'infrastructure'` packages demonstrate the limitation of mutation testing when applied to framework-heavy code (Aspects/Exceptions), where mutations often result in invalid bytecode or behavior identical to the original.

4.4 Formal Specification (JML)

We successfully annotated the core service methods with JML to document invariants and contracts.

- **Specification Scope:** Annotations were applied to `ShoppingCartService` (methods: `buildCartItems`, `updateTotalPrice`) and `OrderService` (methods: `getShippingAddress`, `buildOrder`).
- **Verification Result:** *N/A (Qualitative Only)*.

While the specifications improve code readability and contract clarity, empirical verification (via `openjml -esc`) was not possible. The project's architecture—relying heavily on compile-time bytecode generation (Lombok) and runtime dependency injection (Spring)—creates an Abstract Syntax Tree (AST) that standard JML tools cannot parse without significant refactoring. Thus, the result for JML is a successful *specification* effort, but a failed *verification* attempt due to toolchain incompatibility.

4.5 Performance Benchmarking (JMH)

We integrated the Java Microbenchmark Harness (JMH) to complement functional correctness with operational efficiency metrics. By stripping away database I/O and network latency, JMH measured pure CPU throughput and allowed for the validation of algorithmic choices and framework overhead.

4.5.1 Data Mapping Overhead (ProductMapperBenchmark). This benchmark quantified the runtime overhead of the code-generated MapStruct mapper compared to manual object construction.

ID	Method	Time (ns)	Error ($\pm$)
B03	100x MapStruct	2176.639	130.092
B04	100x Manual (Base)	2014.624	158.505

Table 3: ProductMapperBenchmark Results (Batch Mapping)

The results demonstrate that MapStruct introduces no significant performance overhead compared to the manual mapping baseline; the difference is within the statistical margin of error ($\approx 8\%$). The dominant time component is Object Creation and Stream Processing (approx. 2000 ns for 100 items), not the mapping logic itself. This confirms that MapStruct provides strong dependability through compile-time checks without runtime performance compromise.

4.5.2 Order Service Search Logic (AddressSearchBenchmark). This benchmark compared the original implementation for finding the preferred customer address with an optimized alternative. The original logic used two separate stream passes, while the optimized version used a single for-loop with an early exit condition.

ID	Method	Time (ns)	Error ($\pm$)
B01	Two-Pass Stream (Base)	55.429	7.669
B02	Single-Pass Loop (Opt.)	14.849	2.045

Table 4: AddressSearchBenchmark Results (Search Optimization)

The baseline two-pass stream logic (B01) was identified as a clear performance bottleneck. The optimized single-pass for-loop (B02) achieved a performance gain of approximately 73%, running roughly 3.7 times faster. This optimization is achieved by avoiding the CPU overhead associated with list re-iteration and stream processing, leading to an actionable refactoring conclusion.

4.6 CI/CD & Security Pipeline

To enforce dependability standards automatically, we extended the GitHub Actions pipeline to include secret detection, vulnerability scanning, and static security analysis. This phase involved creating a Docker Hub account for artifact management and configuring secure token access via Action Secrets.

- **Secret Detection (GitGuardian):** A repository scan confirmed a clean history with no hardcoded credentials, validating the project’s configuration management strategy.
- **Vulnerability Management (Snyk):** We integrated Snyk to scan dependencies. To resolve environment isolation issues where the Docker container could not access the JDK 17 setup, we reconfigured the pipeline to execute the Snyk CLI directly on the runner. The scan initially found 13 vulnerabilities, which were mitigated by upgrading Spring Boot (3.5.3 $\rightarrow$ 3.5.6) and commons-lang3 (3.18.0). A single vulnerability remains in lz4-java (v1.8.1); as this is the latest version available on Maven Central, we configured the pipeline to proceed (continue-on-error).

- **Static Security (SonarCloud):** The analysis flagged CorsConfig for wildcard permissions (/**). This was marked as a false positive, as the headless architecture requires open API access for testing. Additionally, we hardened the supply chain by pinning 7 GitHub Action dependencies to their immutable SHA hashes rather than mutable version tags.

5 Discussion & Developer Insights

The process of retrofitting dependability measures onto the *ecommerce-api-spring-kafka* repository revealed distinct challenges and trade-offs inherent in modern Java development.

5.1 The Framework vs. Verification Conflict

A major insight from this study is the friction between “developer productivity” tools and “formal verification” tools.

- **Lombok & JML Incompatibility:** The project utilizes Project Lombok to reduce boilerplate (getters, setters, builders). While this improves code readability, it obscures the source code from JML tools, which expect plain Java syntax. We found that formal verification (using OpenJML) is effectively impossible in a heavy-Lombok environment without delomboking the code first—a step that complicates the build pipeline significantly.
- **Implication:** Teams must decide early: prioritize rapid prototyping (Lombok) or verifiable correctness (JML). Retrofitting verification later is cost-prohibitive.

5.2 Mutation Testing: Signal vs. Noise

Our PiTest integration demonstrated that mutation testing is valuable but requires aggressive tuning.

- **The “Noise” Problem:** Initially, framework code (AOP aspects, Exception classes) generated hundreds of unkillable mutants. For example, mutating an Exception class constructor rarely changes program behavior, leading to false negatives in the report.
- **The Solution:** We found that restricting mutation testing strictly to the domain packages provided high-value insights (improving coverage from 60% to 77%), whereas running it on ‘infrastructure’ layers was computationally expensive and analytically low-yield.

5.3 Performance Baselines vs. System Latency

The integration of JMH microbenchmarks highlighted the distinction between algorithmic efficiency and system reliability, while simultaneously revealing optimization opportunities.

- **Algorithmic Optimization Value:** The AddressSearchBenchmark demonstrated that JMH is crucial for identifying high-leverage algorithmic faults that are difficult to spot through profiling of the full system stack. The subsequent refactoring led to a proven 73% performance improvement in a critical order processing path.
- **The Microbenchmark Limit:** Despite its value, JMH intentionally isolates code from database and network I/O. While it successfully established a baseline for domain logic, it cannot address system-level failures common in distributed

architectures, where I/O latency is often the primary source of operational issues.

- **Strategic Role:** JMH is therefore most valuable as a regression guard for pure *logic* changes—ensuring that the addition of defensive programming or dependability checks (e.g., input validation) does not degrade CPU throughput—but it cannot substitute for end-to-end load testing.

5.4 Infrastructure & Supply Chain Rigor

We observed that dependability extends beyond source code into the build environment itself.

- **Reproducibility:** The initial inability to run the application due to unpinned Docker images proved that build reproducibility is the prerequisite for reliability.
- **Supply Chain Security:** SonarCloud analysis prompted the transition from mutable tags (e.g., `actions/checkout@v3`) to immutable commit SHAs for GitHub Actions. This mitigates the risk of supply chain attacks, ensuring that the pipeline remains dependable even if an external dependency is compromised.

5.5 The Limits of Automated Security

Integrating Snyk and SonarCloud highlighted the gap between “theoretical security” and “pragmatic operations.”

- **Contextual False Positives:** SonarCloud flagged the API’s open CORS policy as a critical hotspot. However, in a headless microservice architecture used for testing, this openness is a feature, not a bug. Automated tools lack the architectural context to make this distinction.
- **Dependency Dead Ends:** Snyk identified a vulnerability in the `lz4-java` library (v1.8.1). However, the remediation (upgrade to v1.10.1) was impossible as Maven Central had not yet indexed the newer version. This forced a trade-off: block the pipeline for a theoretical risk, or configure a waiver (continue-on-error) to maintain development velocity. We chose the latter, documenting the risk rather than stalling the process.

6 Conclusion

This paper presented a case study on hardening an existing open-source microservices application. By establishing a stabilized Docker environment and a GitHub Actions CI pipeline, we created a reproducible foundation for quality assurance.

Our empirical results show that while standard unit testing provided a starting point, it left gaps in branch coverage (62.5% covered). The introduction of Mutation Testing was critical in exposing these gaps, driving improvements in the domain layer where semantic logic resides. Complementing this, the integration of JMH microbenchmarks established a performance baseline, ensuring that increased rigor in logic verification did not come at the cost of computational throughput. Conversely, the attempt to introduce formal verification via JML highlighted a significant toolchain gap: modern frameworks like Spring Boot and Lombok are currently resistant to standard verification tools due to their reliance on runtime injection and bytecode generation.

Furthermore, the integration of security auditing tools (Snyk, GitGuardian, SonarCloud) demonstrated that dependability extends beyond code logic to include supply chain security and configuration management. However, these tools proved not to be “set and forget” solutions; they require contextual interpretation to distinguish between genuine risks (e.g., vulnerable dependencies) and false positives (e.g., intentional CORS configurations).

We conclude that for existing “modern” Java applications, a hybrid approach combining **Containerization**, **Targeted Mutation Testing**, **Performance Baselineing**, and **Automated Security Auditing** offers the highest return on investment. While formal verification remains a specialized technique for algorithmic cores, the pragmatic path to dependability in web architectures lies in a rigorous, automated, and security-aware CI/CD pipeline.

References

- [1] Matthias Gerhard and Max Timur Sönmez. 2025. `ecommerce-api-spring-kafka` Repository. <https://github.com/matiso23/ecommerce-api-spring-kafka>.